Machine Learning - Assignment 2

M.Tech (AIML) | Wisconsin Diagnostic Breast Cancer classifiers

Student name: [TO COMPLETE] **Student ID:** [TO COMPLETE]

Status: Local review draft. Do not submit with placeholders. Personal interpretation and genuine BITS Virtual Lab execution evidence are still required.

1. GitHub repository link

Open Cgreate/StemLight

https://github.com/Cgreate/StemLight

The repository contains the deployed app, training source, saved models and test data. The latest notebook safety fixes, README updates and this revised draft remain local until the user approves a push.

2. Live Streamlit application link

Open the live classification explorer

https://stemlight-adzva7eonekvappgc3xpzg.streamlit.app/

The app supports classifier selection, bundled test data or CSV upload, six evaluation metrics, confusion matrix, classification report and downloadable predictions.

Review item	Current status
Experiment	Five saved classifiers; 455 training rows / 114 held-out test rows
Training source	Verified locally against bundled models; not yet verified on BITS Lab
BITS Lab evidence	Missing - genuine screenshot required on the next page
Student review	Name/ID and personal interpretation incomplete
GitHub / portal	No new push approved; no portal submission performed

3. BITS Virtual Lab execution screenshot

EVIDENCE PLACEHOLDER - NO SCREENSHOT PROVIDED

Insert one genuine screenshot captured while executing the assignment on the actual BITS Virtual Lab. This page does not claim that BITS Lab execution has occurred.

Genuine BITS Virtual Lab screenshot goes here. Do not replace this area with a local-terminal or online-Jupyter screenshot.

Capture checklist

- Show the actual BITS Virtual Lab context and a readable execution result.
- Run `python train_models.py --output-dir experiments/reproduced` in that environment.
- Include the printed five-model metrics and training/test counts when readable.

Execution date / environment / student confirmation: [TO COMPLETE]

4. Complete README content - local draft

StemLight - Machine Learning Classification Explorer

Local review draft for AIML Machine Learning Assignment 2. The student must review the implementation, complete the personal interpretation below, run it on BITS Virtual Lab, and provide genuine execution evidence before final submission. AI assistance must remain within the course's permitted learning-support policy.

a. Problem statement

Build and compare five binary classifiers that distinguish malignant and benign samples in the Wisconsin Diagnostic Breast Cancer dataset. Evaluate every classifier on the same held-out test data using accuracy, AUC, precision, recall, F1, and Matthews correlation coefficient (MCC). Provide an interactive Streamlit application that loads saved models and evaluates test CSVs without retraining on uploaded rows.

b. Dataset description

Property	Description
Dataset	Breast Cancer Wisconsin (Diagnostic), also called WDBC
Public source	UCI Machine Learning Repository
Loader	scikit-learn's `load_breast_cancer(as_frame=True)`, a bundled copy of the UCI dataset
Instances	569 total; satisfies the minimum of 500
Input features	30 numeric features; satisfies the minimum of 12
Target	Binary: `0 = malignant`, `1 = benign`
Class distribution	212 malignant and 357 benign samples
Missing values	None in the bundled dataset
Data split	Stratified 80/20 split, `random_state=42`: 455 training rows and 114 test rows
Test class distribution	42 malignant and 72 benign samples
Preprocessing	StandardScaler inside the Logistic Regression, KNN and SVM pipelines; fitted on training rows only

The features describe cell-nucleus measurements obtained from digitized fine-needle aspirate images. They cover radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension, with mean, standard-error and worst-value summaries. The identifier is not an input feature.

Dataset attribution: Wolberg, Mangasarian, Street and Street (1993), Breast Cancer Wisconsin (Diagnostic), UCI Machine Learning Repository. Dataset DOI. License: CC BY 4.0.

c. GitHub repository and live application

- GitHub repository

- Live Streamlit application

- Streamlit entry point: `app.py`; deployment branch: `main`; cloud Python version: 3.12.

The deployed Streamlit app and training source are already on GitHub. The latest local notebook, documentation and PDF-draft changes require the user's approval before they are pushed.

d. Models used and evaluation

All five models use the same training/test split. Models use scikit-learn 1.6.1 with no hyperparameter search or cross-validation in this experiment.

Model	Configuration
Logistic Regression	StandardScaler; LogisticRegression with `max_iter=2000`, `random_state=42`; other defaults
Decision Tree	DecisionTreeClassifier with `random_state=42`; other defaults, including no depth limit
KNN	StandardScaler; KNeighborsClassifier defaults: 5 neighbors, uniform weights and Euclidean distance
Gaussian Naive Bayes	GaussianNB default parameters
Support Vector Machine	StandardScaler; SVC with RBF kernel, `C=1`, `gamma='scale'`, `probability=True`, `random_state=42`

Precision, recall and F1 use **benign (1)** as the positive class. AUC uses the predicted probability of class 1. Accuracy and MCC use predicted class labels. These conventions match the deployed app; recall in this table is recall for benign samples.

The table below was verified against the five saved models using the repository's `test_data.csv`. Values are rounded to four decimal places; `metrics.csv` contains the unrounded values.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9825	0.9954	0.9861	0.9861	0.9861	0.9623
Decision Tree	0.9123	0.9157	0.9559	0.9028	0.9286	0.8174
KNN	0.9561	0.9788	0.9589	0.9722	0.9655	0.9054
Gaussian Naive Bayes	0.9386	0.9878	0.9452	0.9583	0.9517	0.8676
Support Vector Machine	0.9825	0.9950	0.9861	0.9861	0.9861	0.9623

Quantitative observations for student review

These are factual comparisons from this test split, not a claim of independent student authorship or a substitute for the student's own explanation.

ML Model Name	Observation about model performance
Logistic Regression	Correctly classifies 112/114 test rows. It ties SVM on accuracy, precision, recall, F1 and MCC, and has the highest observed AUC (0.9954).
Decision Tree	Correctly classifies 104/114 rows. It has the lowest observed accuracy, AUC, F1 and MCC among the five models on this split. No training-score evidence is provided to conclude overfitting.
KNN	Correctly classifies 109/114 rows. Its accuracy (0.9561), F1 (0.9655) and MCC (0.9054) exceed Gaussian Naive Bayes and Decision Tree, while its AUC is below Gaussian Naive Bayes.

ML Model Name	Observation about model performance
Gaussian Naive Bayes	Correctly classifies 107/114 rows. Its AUC (0.9878) exceeds KNN's despite lower accuracy and F1; ranking quality and the default decision rule measure different aspects of performance.
Support Vector Machine	Correctly classifies 112/114 rows and ties Logistic Regression on the label-based metrics. Its AUC (0.9950) is slightly lower on this test split.
Overall winner for this test split	Logistic Regression if AUC is used to break the accuracy/F1/MCC tie with SVM. The AUC difference is small; this single split does not establish a statistically significant or universal winner.

Student interpretation - TO COMPLETE: After reproducing the experiment, explain in your own words what you learned, how scaling affects the chosen models, why the observed rankings may differ across metrics, and the limitations of this split. Verify the factual observations above before including them in your submission.

Reproduce training in standard Python / BITS Virtual Lab

The new `train_models.py` uses standard scikit-learn and does not depend on browser-only `piplite` or Jupyter widgets. It trains the five classifiers, saves their preprocessing pipelines, and exports a held-out CSV, evaluation table and training metadata.

Use Python 3.12 and install the pinned dependencies:

```
python -m pip install -r requirements.txt
python train_models.py --output-dir experiments/reproduced
```

Run these commands in the **actual BITS Virtual Lab** after transferring or cloning the project there. The script prints dataset dimensions, training/test counts, all six metrics for each model, and the output location. Capture one genuine screenshot showing execution in that environment. Local execution and online Jupyter execution do not establish BITS Lab execution.

Outputs are written to a separate folder so the models used by the live app are preserved. The script refuses to overwrite existing outputs; choose a fresh output directory for another run. To test the newly produced model set independently, place a copy of `app.py` beside that output folder's `model/` and `test_data.csv` and launch that copy.

Run the Streamlit application

```
python -m streamlit run app.py
```

The default view evaluates the bundled held-out test CSV. Select **Upload CSV** to evaluate another test CSV containing the same 30 feature columns and a `target` column. The app validates inputs, loads the selected saved model, displays the six metrics plus confusion matrix and classification report, and provides downloadable predictions. It never calls the training script or fits a model on uploaded data.

The frontend includes CSV upload, classifier dropdown, all six evaluation metrics, an all-model comparison table, a Matplotlib/Seaborn confusion-matrix heatmap, classification report, and downloadable comparison/prediction CSVs. All five models are evaluated on the same selected test rows.

Notebook safety testing

`main.ipynb` is retained with both dashboards. Run its setup cell in a standard Jupyter environment, then the two dashboard cells. The models are trained once per kernel and reused by both dashboards; restart the kernel to train again. The simple dashboard keeps its own widget state so CSV uploads in the second dashboard do not change its test data.

Both dashboards validate numeric features, reject empty or non-finite inputs, and validate target values before converting them to integers. Decimal, missing, or out-of-range target values are rejected. The CSV dashboard retains a feature-only prediction preview when `target` is absent; metrics require the target column. The deployed Streamlit assignment frontend always requires `target` for uploaded test CSVs.

The notebook uses `ipywidgets` installed by its setup cell. Streamlit's deployed frontend uses the packages in `requirements.txt`, including Matplotlib and Seaborn. Notebook execution and local safety tests do not constitute BITS Virtual Lab execution evidence.

Repository files

File or folder	Purpose
`app.py`	Streamlit frontend and saved-model evaluation
`train_models.py`	Standard-Python training and evaluation source
`model/`	Five saved trained classifiers, including fitted scalers where needed
`requirements.txt`	Pinned deployment and training dependencies
`test_data.csv`	114 held-out test rows, 30 inputs plus target
`metrics.csv`	Six metrics per model, verified from bundled saved models
`training_info.json`	Metadata for the bundled models and split
`main.ipynb`	Preserved Jupyter safety-test notebook with a simple dashboard and CSV dashboard; shares training results within one kernel
`README.md`	Problem, dataset, results, observations and reproduction instructions

Submission status

Prepare one final PDF in this order: GitHub repository link, live Streamlit app link, one genuine BITS Virtual Lab execution screenshot, and the complete finalized README content. The current PDF is a review draft with a screenshot placeholder. Student name/ID, personal interpretation, BITS Lab execution evidence and final review remain outstanding.

The assignment states that AI tools are allowed only for learning support, not direct copy-paste submissions. Review, understand and independently develop/adapt the work within that policy. Do not present generated material or local execution as independent BITS Lab work. The published deadline is 8 November 2026 at 23:59; this draft does not establish an extension, late acceptance or a completed portal submission.